POLYGLOT MEET

Real-Time Speech Translation for Video Conferencing

Business Requirements Document (BRD)

&

Product Requirements Document (PRD)

Version 0.1 — Draft

Prepared: July 13, 2026

Status: Prototype / Concept Stage

Document Control

Version History

Version	Date	Author	Summary of Changes
0.1	13-Jul-2026	Product Owner	Initial draft combining BRD + PRD for prototype scoping

Distribution / Audience

- Founder / Product Owner
- Engineering (Backend – Go, Frontend – JS/WebRTC)
- Design (UX for call interface)
- Prospective investors / early advisors (if applicable)

Executive Summary

Polyglot Meet is a real-time video conferencing application that removes the language barrier from live conversation. Each participant selects the language they want to hear, and speech from every other participant is transcribed, translated, and rendered back to them — either as live captions or as a dubbed voice track — with minimal delay, similar to how a live news broadcast is simultaneously translated for a foreign audience.

The product is being scoped as a prototype first, built on a Go backend (using Pion for WebRTC), a lightweight vanilla-JS frontend, and third-party AI APIs (OpenAI or Gemini) for speech-to-text, translation, and text-to-speech. This document captures both the business rationale (BRD) and the detailed functional/technical requirements (PRD) needed to guide a phased build, starting with live translated subtitles before layering on voice dubbing and voice cloning.

Aspect	Summary
Problem	Multilingual teams and communities cannot meet naturally on video without a human interpreter or manual subtitles.
Solution	In-call, per-listener, real-time translated captions and/or dubbed audio, selectable from a simple dropdown.
Target Users	Remote/distributed teams, international communities, education, customer support, healthcare consults.
Differentiator	Per-listener language choice (not one global language for the whole call) + optional voice-dubbing, not just subtitles.
V1 Scope	1:1 and small group calls, live translated subtitles only, 2–4 language pairs.
Tech Stack	Go + Pion (WebRTC), vanilla JS frontend, OpenAI/Gemini APIs for STT, translation, TTS.

Part A — Business Requirements Document (BRD)

A.1 Business Objectives

The business objectives define why this product should exist and what it should achieve in the first 6–12 months.

- Validate that real-time, per-listener translation in a video call is technically feasible at acceptable latency (target: under 3 seconds end-to-end for prototype).
- Prove demand with a working prototype that can be demoed to 10–20 real users / teams.
- Establish a cost model for AI API usage per minute of call time, per language pair.
- Create a foundation that can later support monetization (subscription per host, per-minute billing, or enterprise licensing).

A.2 Business Problem / Opportunity

Existing video conferencing tools (Zoom, Google Meet, Teams) either offer no live translation, or offer only text captions in a single language for the whole meeting, requiring everyone to agree on one output language. There is no mainstream product offering:

- Per-listener language selection (each person hears/reads their own chosen language, independent of others).
- Real-time voice dubbing (not just subtitles) that preserves the natural flow of conversation.

This is the opportunity: a video calling experience where language stops being a barrier to natural conversation, targeted first at distributed teams, cross-border businesses, international classrooms, and community/NGO use cases.

A.3 Stakeholders

Stakeholder	Role / Interest
Product Owner / Founder	Overall vision, prioritization, prototype funding
Backend Engineer(s)	Go/Pion WebRTC pipeline, AI API orchestration, infra
Frontend Engineer(s)	Call UI, language selector, subtitle/audio rendering
End Users — Hosts	Start/manage calls, invite participants
End Users — Participants	Select target language, consume translated output
AI API Providers	OpenAI / Google Gemini — STT, translation, TTS services
Early Adopter Teams	Pilot testing, feedback on latency & translation quality

A.4 Scope

In Scope (Prototype / Phase 1–3)

- 1:1 and small-group (up to ~4–6 participants) video calls
- Per-participant target language selection via dropdown
- Live translated captions (Phase 1)
- Live translated dubbed audio with generic TTS voice (Phase 2)
- Voice-cloned dubbed audio + audio ducking (Phase 3)

Out of Scope (for now)

- Large webinar / broadcast-scale calls (50+ participants)

- Recording, transcript export, or meeting-summary features
- Mobile native apps (iOS/Android) — web-first only
- Billing / payment integration
- Enterprise SSO, compliance certifications (HIPAA, SOC 2) — revisit if healthcare/enterprise use case is pursued

A.5 Success Metrics (Business KPIs)

Metric	Target for Prototype Phase
End-to-end translation latency	< 3 seconds (subtitles), < 5 seconds (dubbed audio)
Pilot users onboarded	10–20 individuals / 3–5 teams
Call completion rate	> 80% of started calls run to natural completion without crash
Translation intelligibility (user-rated)	> 7/10 average in post-call survey
Cost per call-minute (AI APIs)	Tracked and modeled; target ceiling set once real usage data available

A.6 Assumptions & Constraints

Assumptions

- Users have a stable broadband connection (minimum ~1 Mbps up/down).
- Users are willing to tolerate some latency in exchange for cross-language communication.
- Third-party AI APIs (OpenAI/Gemini) will remain available and reasonably priced for prototype-scale usage.

Constraints

- No Python in the stack — backend is Go-only by decision.
- Small team / solo builder — prototype must stay lean and avoid premature scaling work.
- Dependent on third-party AI vendor rate limits, pricing, and latency — not self-hosted models at this stage.

A.7 Risks

Risk	Impact	Mitigation
Latency makes conversation feel unnatural	High	Start with streaming STT + subtitle-only MVP; optimize before adding TTS layer
AI API costs scale faster than expected with multiple languages per call	Medium	Model cost per participant-language early; consider caching / rate-limiting concurrent translations
Echo / feedback loop (translated audio re-captured by mic)	Medium	Strong echo cancellation + option to route translated audio to headphones only
Overlapping speech breaks STT accuracy	Medium	V1 constraint: encourage one active speaker at a time; add diarization later
Vendor lock-in to OpenAI/Gemini pricing or policy changes	Low–Medium	Abstract AI calls behind an internal interface so providers can be swapped

A.8 Budget & Cost Considerations

For prototype phase, primary costs are: (1) AI API usage — billed per audio-minute for STT/TTS and per-token for translation; (2) compute hosting for the Go backend and WebRTC media relay (TURN/STUN servers); (3) optional voice-cloning API costs in Phase 3, which are typically priced at a premium over standard TTS.

Recommendation: instrument cost-per-call-minute tracking from day one so the cost model is based on real data rather than estimates before any pricing or monetization decisions are made.

Part B — Product Requirements Document (PRD)

B.1 Product Overview

Polyglot Meet is a browser-based video calling product. Each participant joins a call and selects, from a simple dropdown, the language they want to receive. As other participants speak, their speech is transcribed, translated into each listener's chosen language, and delivered back either as real-time captions (Phase 1) or as translated dubbed audio (Phase 2–3), with the original speaker's audio ducked underneath for naturalness.

B.2 Goals

- Enable two or more people who don't share a language to hold a natural real-time conversation over video.
- Keep translation latency low enough that the conversation still feels responsive.
- Make language selection effortless — a single dropdown, no configuration required.
- Ship an initial working prototype quickly by sequencing subtitles → generic TTS → voice cloning.

Non-Goals (for current phase)

- Perfect, publication-quality translation accuracy (good-enough real-time accuracy is the bar, not literary precision).
- Supporting every world language on day one — start with a small, high-value set (e.g., English, Spanish, Hindi, French, Mandarin).
- Supporting large-scale webinars or one-to-many broadcast translation.

B.3 User Personas

Persona	Description	Primary Need
Remote Team Lead	Manages a distributed team across 2–3 countries with different native languages	Run standups/meetings without a human interpreter
International Student	Studying or collaborating with peers who speak a different language	Follow group discussions in real time
Small Business Owner	Talks to overseas clients/vendors periodically	Occasional, reliable 1:1 translated calls
Community / NGO Coordinator	Runs multilingual community calls or support sessions	Inclusive group conversations across languages

B.4 Key Use Cases / User Stories

1. As a participant, I want to select my preferred listening language from a dropdown before or during a call, so that I hear/read everything in a language I understand.
2. As a speaker, I want to speak naturally in my own language without needing to change anything, so that translation happens transparently for others.
3. As a listener, I want to see live translated captions of what others are saying, so that I can follow along even before audio dubbing is available.
4. As a listener (Phase 2+), I want to hear a translated voice instead of reading captions, so that the experience feels like a natural conversation, not a subtitle feed.
5. As a listener, I want to know when a translation is in progress (a visual indicator), so that I understand why there's a short delay after someone speaks.
6. As a host, I want to start a call and share a link, so that others can join without account setup friction (for prototype).

B.5 Functional Requirements

Phase 1 — Live Translated Subtitles (MVP)

ID	Requirement	Priority
FR-1.1	System shall capture each participant's audio via WebRTC in real time.	Must
FR-1.2	System shall stream captured audio to a speech-to-text (STT) service using streaming/partial-result mode.	Must
FR-1.3	System shall allow each participant to select a target language from a dropdown, independent of other participants' choices.	Must
FR-1.4	System shall translate recognized text into each listener's selected target language in near real time.	Must
FR-1.5	System shall display translated captions on-screen, attributed to the correct speaker.	Must
FR-1.6	System shall show a lightweight 'translating…' indicator while a translation is in flight.	Should
FR-1.7	System shall support at least 4–5 initial languages (e.g., English, Spanish, Hindi, French, Mandarin).	Must

Phase 2 — Dubbed Audio (Generic Voice)

ID	Requirement	Priority
FR-2.1	System shall convert translated text into speech (TTS) using a stock/generic voice per language.	Must
FR-2.2	System shall play translated audio to each listener according to their selected language.	Must
FR-2.3	System shall duck (lower) the original speaker's audio volume while translated audio plays.	Should
FR-2.4	System shall prevent translated audio from being re-captured by the listener's own microphone (echo prevention).	Must
FR-2.5	System shall allow a listener to toggle between 'captions only' and 'dubbed audio' modes.	Should

Phase 3 — Voice Cloning & Polish

ID	Requirement	Priority
FR-3.1	System shall optionally generate translated speech using a cloned approximation of the original speaker's voice.	Could
FR-3.2	System shall allow a host/participant to opt in or out of voice cloning for privacy reasons.	Must (if 3.1 shipped)
FR-3.3	System shall support graceful fallback to generic TTS voice if cloning fails or is disabled.	Must (if 3.1 shipped)
FR-3.4	System shall use available visual context (e.g., video frames) to improve translation disambiguation where supported by the AI provider.	Could

B.6 Non-Functional Requirements

Category	Requirement
Performance / Latency	End-to-end translation latency target: < 3s for captions, < 5s for dubbed audio, measured from end-of-utterance to delivery.
Scalability	Prototype should comfortably support small group calls (up to ~6 participants); horizontal scaling deferred to later phase.
Reliability	Call should degrade gracefully (e.g., fall back to captions only) if TTS or translation service is slow/unavailable, rather than dropping the call.
Security & Privacy	Audio/text sent to third-party AI APIs should not be stored longer than necessary; voice-cloning data (if used) requires explicit consent.
Compatibility	Should work in modern evergreen browsers (Chrome, Edge, Firefox, Safari) supporting standard WebRTC APIs.
Observability	Backend should log per-call latency and API cost metrics for monitoring and cost modeling.

B.7 System Architecture Overview (Reference)

This mirrors the technical approach already discussed and is included here for completeness as the PRD's technical reference point:

- WebRTC media handled server-side via Pion (Go) to allow interception and processing of audio packets.
- Signaling and real-time translated captions delivered via WebSockets.
- STT, translation, and TTS calls made to OpenAI and/or Google Gemini APIs, abstracted behind an internal Go interface so providers can be swapped.
- Frontend: plain HTML/CSS/vanilla JS using the browser's native RTCPeerConnection and MediaRecorder APIs — no heavy framework for the prototype.
- Deployment via Docker containers for the Go backend; TURN/STUN infrastructure for NAT traversal.

B.8 UX / Interaction Requirements

- Language dropdown should be visible and changeable at any point during a call, not just at join time.
- Captions should be visually distinct per speaker (e.g., name label + caption bubble).
- A subtle 'translating…' affordance should appear near a speaker's video tile during processing, to set the listener's expectation for the delay.
- Audio/caption mode toggle should be a single, obvious control — not buried in settings.

B.9 Analytics & Success Metrics (Product-Level)

Metric	Why it matters
Average end-to-end translation latency per call	Directly measures the core UX promise of the product
Caption vs. dubbed-audio mode usage split	Indicates which experience users actually prefer
Language pairs used most frequently	Guides which languages to prioritize/optimize first
Drop-off / call abandonment rate	Signals whether latency or quality issues are breaking the experience
Post-call intelligibility rating	Direct user-reported quality signal, complements objective latency metrics

B.10 Release Plan / Roadmap

Phase	Focus	Exit Criteria
Phase 1 — Subtitles MVP	WebRTC capture, streaming STT, translation, on-screen captions	Two users on different languages can hold a captioned conversation with < 3s caption delay
Phase 2 — Dubbed Audio	Generic TTS playback, audio ducking, echo prevention	Users can choose dubbed-audio mode with acceptable naturalness and no feedback loops
Phase 3 — Voice Cloning & Polish	Voice cloning, visual-context translation, UX refinement	Opt-in cloned-voice dubbing works reliably with graceful fallback

B.11 Open Questions / Future Considerations

- Symmetric vs. per-speaker-pair language selection — does everyone pick one language, or can a listener pick a different language per speaker?
- How will the product handle more than one person speaking at once (overlap/diarization)?
- What is the long-term monetization model — per-minute billing, subscription per host, or enterprise licensing?
- Should the product eventually support self-hosted/open-source STT and TTS models to reduce dependency on third-party API pricing?
- Will recording / transcript export become a requested feature once the core experience is validated?

Appendix — Glossary

Term	Definition
STT	Speech-to-Text — converting spoken audio into written text
TTS	Text-to-Speech — converting written text into spoken audio
WebRTC	Web Real-Time Communication — browser standard for peer-to-peer audio/video/data
Pion	An open-source WebRTC implementation written in Go
VAD	Voice Activity Detection — detecting when someone is speaking vs. silent
Audio Ducking	Automatically lowering one audio track's volume so another can be heard clearly
Diarization	Identifying and separating who is speaking when multiple voices are present
Voice Cloning	Generating synthetic speech that mimics a specific person's vocal characteristics